

Meridian — Daily Work Plan

12 weeks, day by day.

Author: Saeed Hussain

Written: 17 September 2026

Time per day: 2 to 3 hours

Days per week: 6 working days, 1 rest day

How to use this plan

Each day has one task and a "**Done when**" line.

The "Done when" line is the important part. It tells you when to stop. If you can say yes to it, the day is finished. Close the laptop. Do not keep going.

The days are numbered 1 to 6, not named. Start on any day of the week you like. Day 7 is your rest day. Take it. A tired brain writes bugs into merge algorithms, and those bugs are very hard to find later.

Five rules

- 1. Commit every day.** Even small work. The history is part of the project.
 - 2. Write the test before moving on.** If today's work has no test, it is not done.
 - 3. Three-day limit.** If one problem eats three days, build a simpler version and move on. Come back later.
 - 4. Do not add features.** If you get an idea, write it in `IDEAS.md` and keep going. This is the main reason projects die.
 - 5. If you fall behind, cut work, not quality.** Being one week late is fine. Shipping something broken is not.
-

The 12 weeks at a glance

WEEK	GOAL	WHY IT MATTERS
1	Set up the project	A clean base means no fighting your tools later
2	IDs and clocks	Everything in the algorithm stands on this
3	Typing letters together	First real merging
4	Deleting, and proving it works	The week that makes or breaks the project
5	Other data types	Titles, lists and settings also merge
6	Saving to disk	Work survives crashes
7	Computers talk to each other	The first live demo
8	Bad networks	Works when the internet is poor
9	The editor screen	It starts to look like a real app
10	Offline and history	The main promise of the project
11	Encryption and desktop app	Private, and installable
12	Speed, docs, release	Finish it properly

Week 1 — Set up the project

Goal: everything runs, tests run, nothing is built yet.

DAY	TASK	DONE WHEN
1	Make the folders: <code>packages/core</code> , <code>apps/web</code> , <code>apps/desktop</code> . Start git. Set up npm workspaces.	<code>npm install</code> works from the top folder
2	Create the Next.js app in <code>apps/web</code> .	The page opens in the browser
3	Add <code>jsconfig.json</code> with <code>checkJs</code> on. Add ESLint.	Your editor shows an error when you pass a wrong value
4	Set up the test runner using Node's built-in <code>node:test</code> . Write one small test.	<code>npm test</code> passes
5	Put the repo on GitHub. Add CI that runs lint and tests on every push.	The tick mark shows green on GitHub
6	Write the README: what Meridian is, in five lines. Read about Lamport clocks.	A stranger could read the README and understand the goal

Week 2 — IDs and clocks

Goal: every change gets a name that is unique forever.

This week has no visible result. Do not worry. This is the foundation, and the rest of the project stands on it.

DAY	TASK	DONE WHEN
1	Give each device a random ID when it first runs. Save it.	The same device keeps its ID after a refresh
2	Add a counter that goes up by one for every change. Combine device ID + counter to make a change ID.	Two devices can never make the same ID
3	Decide the shape of a change (an "operation"): its ID, its type, its value, where it goes. Write it down with JSDoc comments.	The shape is documented in <code>docs/DESIGN.md</code>
4	Write the compare function that puts two IDs in order.	Tests prove the order is always the same, whichever way round you ask
5	Build the operation log: add a change, read all changes.	Tests pass for adding and reading
6	Build the "state vector": a small record of what you have seen from each device.	You can ask "what am I missing?" and get the right answer

Week 3 — Typing letters together

Goal: two copies of a document, both typing, same result.

DAY	TASK	DONE WHEN
1	Build the character list. Each letter is an item with an ID and a link to the letter before it.	You can build <code>Hello</code> by hand and read it back
2	Write <code>insert(position, letter)</code> . It makes an operation.	Typing locally produces the right text
3	Write <code>applyRemote(operation)</code> . It places a letter using IDs, not positions.	A change from another device lands in the right place
4	The tie-break rule. When two letters are inserted in the same spot at the same time, order them by ID.	Both replicas agree on the order, every time
5	Write <code>toString()</code> to turn the structure into readable text.	The text matches what you typed
6	Test: two replicas, both insert at the same place, then swap changes.	Both show identical text

Week 4 — Deleting, and proving it works

Goal: the algorithm is correct, and you can prove it.

This is the most important week in the project. Do not rush it.

DAY	TASK	DONE WHEN
1	Add delete. A deleted letter is marked hidden, not removed. This mark is called a tombstone.	Deleted text disappears from view
2	Handle a delete arriving from another device, including for a letter you have not received yet.	No crash, and the result is right
3	Map the visible position to the internal position. Hidden letters must not be counted.	Typing after a deletion lands in the correct place
4	Write the random test generator: it makes thousands of random edits in random orders.	It runs and produces different results each time, from a seed you can repeat
5	Run convergence tests on three replicas, thousands of times.	All three always end identical
6	Fix every bug the random test found.	The test suite runs clean ten times in a row

Stop here and check. If day 6 passes, the hard part of Meridian is solved. If it does not, spend next week fixing it. Do not move on with a broken core. Everything later assumes this works.

Week 5 — Other data types

Goal: titles, tags and settings merge too, not just text.

DAY	TASK	DONE WHEN
1	Build the last-write-wins register, for single values such as a title.	Two devices set the title; both end with the same one
2	Build the add-wins set, for tags. Adding beats removing when they clash.	Concurrent add and remove gives the same answer on both sides
3	Build the map that holds named fields.	You can set and read fields
4	Put it together: a document is a map of fields plus the text.	One object holds the whole document
5	Save the whole state to JSON, and load it back.	Load then save gives the exact same JSON
6	Run the random tests against the new types.	All types converge

Week 6 — Saving to disk

Goal: close the laptop, open it, nothing lost.

DAY	TASK	DONE WHEN
1	Define the storage interface: save a change, load all changes, save a snapshot.	The core does not know or care which storage is used
2	Write the IndexedDB version for the browser.	Changes survive a page refresh
3	Write the SQLite version for the desktop, using <code>better-sqlite3</code> .	The same tests pass on both
4	On start-up, load saved changes and rebuild the document.	Reopening shows your text
5	The crash test: kill the app while typing, then reopen.	At most the last letter is missing, and nothing is broken
6	Snapshots: save the current state and throw away old changes safely.	A long document loads fast

Week 7 — Computers talk to each other

Goal: two browser tabs sync live. This is your first real demo.

DAY	TASK	DONE WHEN
1	Write the small signalling server (Node and WebSocket). It only introduces peers.	Two tabs can find each other
2	Open a WebRTC connection between the two tabs.	The connection reports "connected"
3	Open a data channel and send a test message.	A message typed in one tab appears in the other
4	Exchange state vectors when two peers meet.	Each side knows what the other is missing
5	Send only the missing changes, not everything.	A new peer joining a long document does not download it all twice
6	Milestone. Type in one tab, watch it appear in the other. Record it.	Live typing syncs between two tabs

Week 8 — Bad networks

Goal: it still works when the internet is poor. This is the real test.

DAY	TASK	DONE WHEN
1	Support more than two peers connected at once.	Three tabs all stay in sync
2	Handle disconnects: queue changes, send them on reconnect.	Pulling the cable and plugging it back in loses nothing
3	Handle repeated and out-of-order messages.	Applying the same change twice changes nothing
4	Build the network simulator: add delay, drop messages, shuffle order.	You can turn a bad network on and off from a switch
5	Run the convergence tests through the bad network.	All peers still end identical
6	Fix what broke, and write down what you learned.	Tests pass with 30% packet loss

Week 9 — The editor screen

Goal: it looks and feels like a real app.

DAY	TASK	DONE WHEN
1	Connect the editor box to the document. Typing goes through the core.	Typing works normally
2	Keep the cursor in the right place when a remote change arrives above it.	Your cursor does not jump while someone else types
3	Show who is online.	Names appear and disappear as tabs open and close
4	Show other people's cursors, each with a name and colour.	You can watch someone else type
5	Show the connection state: online, offline, syncing, synced.	The state is always honest
6	Polish: shortcuts, select-all, copy, paste.	Paste of a large block does not freeze the page

Week 10 — Offline and history

Goal: the main promise of the project, working end to end.

DAY	TASK	DONE WHEN
1	Detect going offline. Keep saving locally and queue changes.	Editing offline feels exactly like editing online
2	On reconnect, merge everything and show "synced".	No duplicate text after a long offline session
3	Automate the four-window test.	The test runs by itself and checks all four match
4	Replay history: rebuild the document as it was at any earlier point.	You can view yesterday's version
5	Add the time slider to the interface.	Dragging it scrubs through the document's history
6	Fix bugs. Run every test again.	Everything green

Week 11 — Encryption and the desktop app

Goal: private, and installable on Windows.

DAY	TASK	DONE WHEN
1	Create a key for each document. Put it in the link, after the <code>#</code> , so the server never receives it.	Opening the link opens the document; the server log shows no key
2	Encrypt every change before sending, using the browser's built-in AES-GCM.	Peers still sync correctly
3	Check what the server actually sees.	The server log shows only scrambled data
4	Wrap the app in Electron.	The desktop window opens and works
5	Use SQLite storage inside the desktop app.	The desktop app works with the internet fully off
6	Build the Windows installer.	The <code>.exe</code> installs and runs on a clean machine

Week 12 — Speed, documents, release

Goal: finish it properly. Most people skip this week. Do not.

DAY	TASK	DONE WHEN
1	Test with a document of 100,000 letters. Find what is slow.	You know which function is the problem
2	Fix the slow parts. Measure before and after.	Typing feels instant on the big document
3	Write the documentation for the core package.	Someone else could use it without asking you questions
4	Write the README properly, with a recording of the four-window test at the top.	The demo plays without anyone needing to install anything
5	Deploy the web demo. Publish the installer.	A stranger can try it from a link
6	Write the article explaining how the merge algorithm works.	Published, and honest about what is still weak

When you finish each week

Ask yourself three questions:

1. Do all the tests still pass?
2. Is it still fast?
3. Did I add anything that was not in the plan?

If the answer to the third is yes, think hard about removing it.

If you fall behind

That is normal. Almost everyone does. Cut in this order:

1. **Cut the time slider** (week 10, days 4 and 5). Nice, but not needed.
2. **Cut the desktop app** (week 11, days 4 to 6). The web version proves the same thing.
3. **Cut encryption** (week 11, days 1 to 3). Add it after release, and say so honestly.

Never cut these three:

- The random convergence test (week 4).
- The offline and reconnect work (week 10).
- The demo recording (week 12).

Without those, the project does not prove anything, and proving the hard part is the only reason to build it.

The one sentence to remember

If week 4 works, you have built something most developers only ever install.